

VIETNAMESE-GERMAN UNIVERSITY



IoT and Smart Devices

**Edge AI-Based Predictive Maintenance System for DC Fan Fault
Diagnosis Using Vibration Analysis Project Proposal**

Major: Computer Science & Engineering

Course: IoT and Smart Devices

Instructor: VO BICH HIEN, Prof.

Students:

MAI CHI KIEN - 104240280

NGO HAI THIEN - 104240041

LAY VO DUC - 104240618

---o0o---

TABLE OF CONTENTS

1. Introduction	3
2. Problem Statement	3
3. Project Objectives	3
5. Proposed System Architecture	5
6. Hardware Components	5
7. Software and Development Stack	6
8. Methodology	6
8.1 Data Collection	6
8.2 Signal Preprocessing	7
8.3 TinyML Model Development:	7
8.4 TinyML Deployment	8
9. Web Dashboard and Monitoring	8
10. Expected Outcomes	9
11. Alignment:	9
12. Plan	9

1. Introduction

We often overlook the importance of risk management when operating with electronic devices around the house and in the factory. Therefore, while it is still vital to check in on the physical mechanism of the device, we still need something to alert us of the possible errors that will have happened in the system to reduce cost and prevent unwanted critical failures of the system.

This project aims to look into that problem and create a practical solution. It proposes an Edge AI-based predictive maintenance system for rotating machinery using vibration signal analysis from a low-cost MEMS sensor. The main objective of this project is to detect, analyse and report abnormal operating hazards in a 12mm DC fan.

The project will include the following parts: Physical sensors, local web dashboard

2. Problem Statement

Cooling fans, motors, or industrial blowers are some rotating machinery which is affected by mechanical problems, for example: balance issues and external obstruction. This could lead to some drawbacks if they are not being tackled properly, such as:

- Reduced efficiency
- Louder noise, vibrations
- Mechanical downgrade
- Unexpected breakdown
- Higher maintenance costs

→ Therefore, the goal of this project is to create an affordable, scalable, and real-time predictive maintenance system that can solve the problem directly on embedded hardware.

3. Project Objectives

The main objectives of this project are:

1. Develop a vibration-based predictive maintenance system using MPU-6050.
2. Collect and preprocess vibration data from a DC fan under multiple operating conditions.
3. Train and evaluate machine learning models for fault classification.
4. Deploy the trained model on an ESP32 microcontroller using TinyML techniques.
5. Implement real-time fault monitoring of DC Fan and alerting through a web dashboard.
6. Evaluation includes:
 - Classification accuracy
 - Inference latency

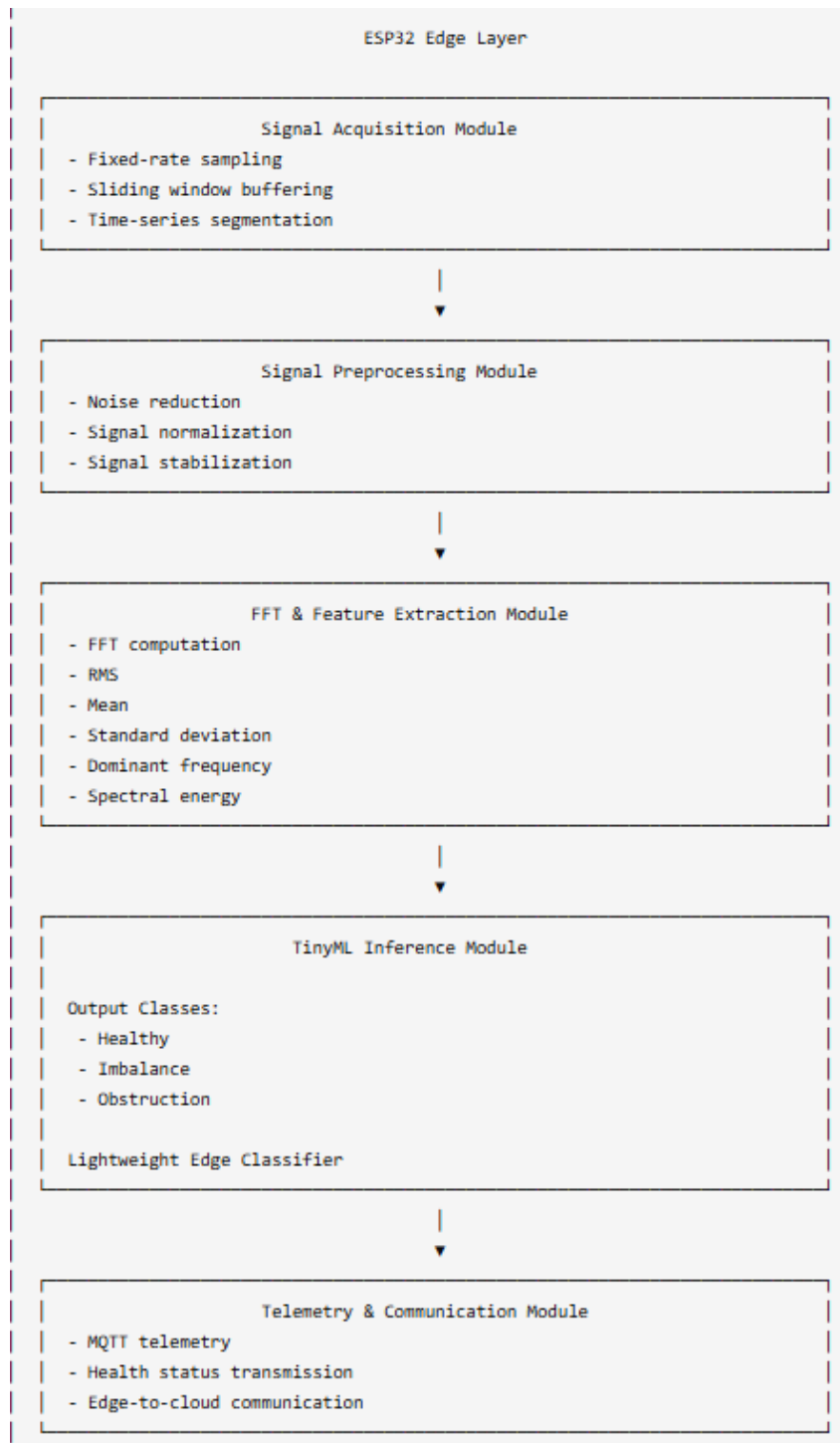
- Memory utilization
- Real-time monitoring capability

4. Target Fault Conditions

The system classifies 3 conditions:

Class	Description
Normal operation – Healthy	Fan operating under normal condition.
Imbalancing fans' blades – Imbalance	Additional object attached to fan blade causing imbalance.
External Mechanical Inference – Obstruction	Artificial obstruction applied to rotating fans.

5. Proposed System Architecture



6. Hardware Components

Component	Purpose
ESP32	Send the collected data to a laptop for training models. Edge AI processing and wireless communication.
MPU-6050	Vibration sensing
12mm DC Fan	Rotating machinery test platform
CP2102 USB Interface	Programming and serial communication
Laptop/Server	Dataset storage and model training

7. Software and Development Stack

Technology	Purpose
Edge Impulse	Dataset management and TinyML deployment
TensorFlow	Neural network training
React	Web dashboard frontend
Nextjs	Backend API and monitoring services
MQTT Protocol	Real-time device communication
Microsoft Azure Ecosystem	IoT-Hub services and scalable infrastructure

8. Methodology

8.1 Data Collection

Raw vibration data will be collected from MPU-6050 under 3 states of a DC fan. It includes normal operation, imbalance operation and external obstruction.

We will develop and design the physical components to improve signal consistency, reduce bias from the environment, and increase the reliability of the dataset.

Raw data include X, Y, Z axes and it will be used to make time-series vibration signals. Then we will do some work on these vibration signals. We will do this to get them ready for our TinyML model.

The vibration signals are taken at a pace. This pace is just right for looking at the vibration spectrum. It is also good for getting features from the data using FFT.

We aim to collect 150 windows per class with diverse conditions of 3 classes.

8.2 Signal Preprocessing

The vibration signals from the MPU6050 sensor are preprocessed before feature extraction and model inference. The preprocessing stage includes:

- Noise reduction and signal stabilization.
- Signal normalization.
- Sliding window segmentation is applied to divide continuous vibration signals into fixed-length analysis windows for feature extraction and TinyML inference
- Time-domain statistical feature extraction
- Frequency-domain analysis using Fast Fourier Transform (FFT)

The extracted features are designed to capture both vibration intensity and spectral characteristics of the DC fan system.

Selected features include:

- Root Mean Square (RMS)
- Mean
- Standard Deviation
- Peak Amplitude
- Dominant Frequency
- Spectral Energy Distribution

8.3 TinyML Model Development:

FFT-based and vibration features which are extracted from the preprocessing stage will be used to train the TinyML model.

To ensure the model is efficient and reliable, we prioritize lightweight machine learning rather than deep learning . The model is designed to identify vibration pattern variations between normal and abnormal operating conditions of the DC fan system.

The training process uses labeled data to train the model. Feature vectors are extracted from both time-domain and frequency-domain analysis to enhance the accuracy and minimize the noise.

Model evaluation criteria includes prediction accuracy, inference consistency and stability across testing implementation . The priority of the evaluation phase is to reduce complexity and the risk of overfitting due to the limited dataset size (Because we use ESP-32 as MCU).

The final TinyML model is optimized for deployment on ESP-32.

8.4 TinyML Deployment

The trained model is deployed on the ESP32 for real-time classification and monitoring.

The deployment pipeline:

- Real-time vibration signal from the MPU6050 sensor
- Signal preprocessing
- FFT-based feature extraction
- On-device TinyML inference
- Send data to the Cloud.

To ensure stable embedded operation, the deployment process prioritizes:

- Low memory consumption (By using ESP32)
- Low inference latency
- Lightweight complexity
- Reliable performance under ESP32 constraints.

The ESP32 handles the data on the device, including model classification. It doesn't send the data to Cloud for evaluation. This means MCU could make decisions quickly without high cost of Cloud platform.

The ESP32 sends the result to Azure through MQTT. Users now could see the conditions of their machines on our web.

9. Web Dashboard and Monitoring

A web-based monitoring platform will be developed to provide:

- Real-time device monitoring
- Fault status visualization

- Historical event logging
- Device condition overview
- Alert notifications

The backend system will use Nextjs APIs and MQTT subscriptions to receive telemetry from the ESP32 nodes.

The frontend dashboard will be implemented using React for responsive visualization and scalability.

10. Expected Outcomes

The goal of this project is to create a Predictive Maintenance system. It is combined with edge computing and data visualization. Results expected are:

- **Custom Vibration Dataset:** A labeled dataset, which includes time capturing unprocessed 3-axis accelerometer data from the DC motor fan across three conditions: Normal, Imbalance, and Obstruction.
- **Optimized Edge AI Model:** An optimized classification model that is quantized and successfully deployed onto the ESP32 microcontroller, making sure memory footprint is minimalized and no memory overflow detected..
- **Real-Time Edge Inference System:** A hardware setup (ESP32 + MPU-6050) is able to preprocess signal on MCU and detect fault with inference latency of under 100ms, operating independently.
- **Monitoring Dashboard:** Develop a website visualizing real-time data, while the dashboard displays the current states, vibration data and TinyML inference outputs, providing alerts for the managers.

11. Alignment:

11.1 Embedded Systems Engineer

Handles everything physical and on-device:

- ESP32 firmware & MPU-6050 integration.
- TinyML deployment on edge device
- MQTT communication set up
- Azure IoT Hub configuration and device connection
- Signal preprocessing on embedded hardware

11.2 ML Engineer:

- Dataset management & data collection pipeline
- Signal preprocessing (FFT, RMS, normalization, windowing)
- TensorFlow model training & evaluation
- Model optimization (quantization, memory reduction)

11.3 Web developer:

- Nextjs backend API & MQTT subscriber
- React web dashboard (real-time charts, fault status, alerts)
- Historical event logging & device condition overview
- Alert notification system

12. Plan

	Week 1	Week 2	Week 3	Week 4	Week 5
Embedded Systems Engineer	-Azure IoT Hub Setup. -MQTT device connection. -Data storage setup.	-Research about Edge Impulse.	-Quantitized model. -Deploy it on ESP-32.	-Deploy TinyML successfully. -Read bugs	-Check the system carefully before representation .
ML Engineer	-Evaluate the signal quality. -Develop preprocessing code -Collect data	-Filter the noise -Train the first model -Collect data	-Optimize the model -Work with the Embedded System Engineer to match the constraints (ESP-32)	-Optimize model to satisfy the constraints.	-Check the system carefully before representation
Web Developer	-Sketch the web. -Help with work.	-Based on data, build logging, visualized features.	-Complete the web with a highly UX-friendly display.	-Help with other work.	
Team	Hardware setup. Support if necessary	Support if necessary.			